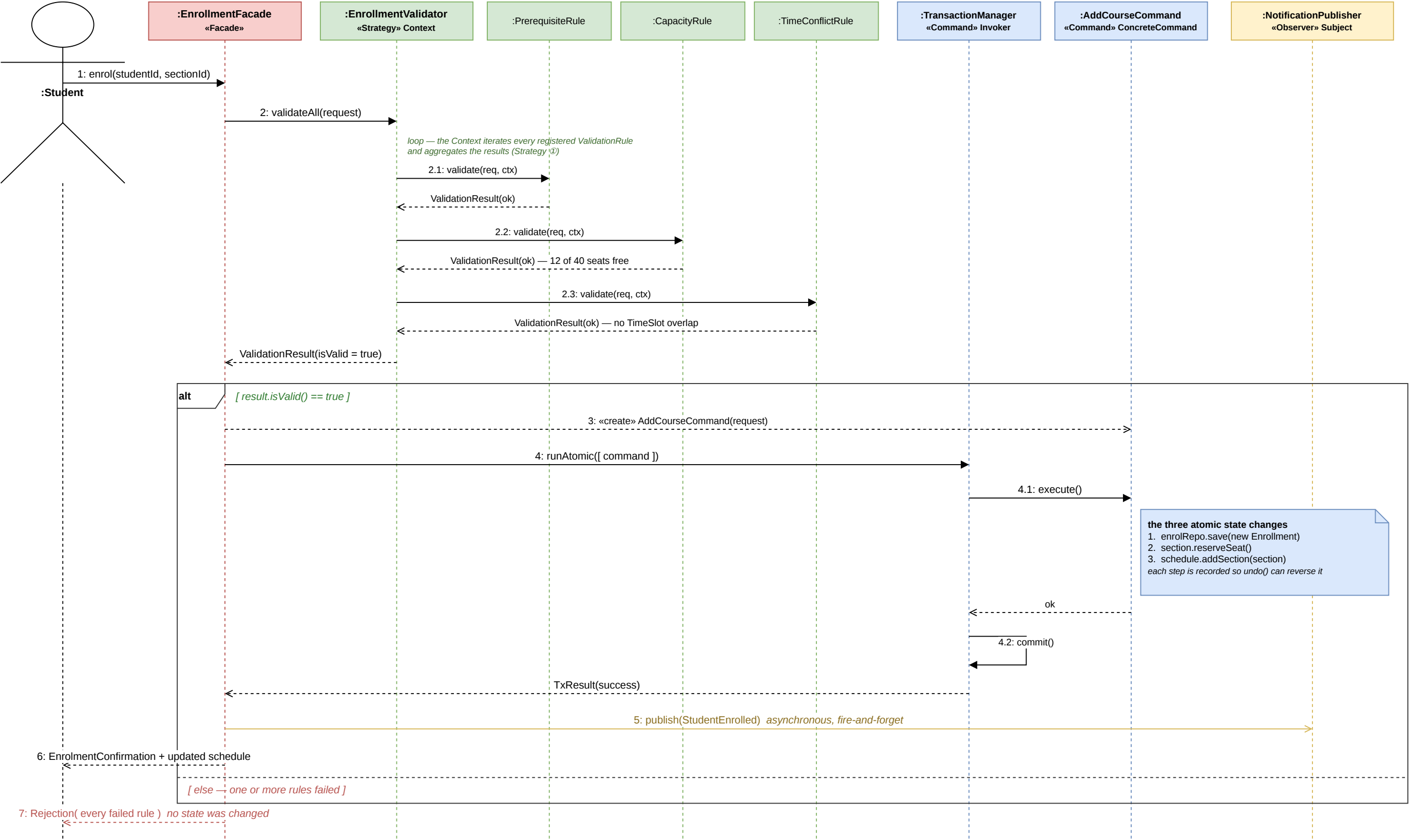


Sequence Diagram — Student Enrols in a Course

Use case from §3.1: "check prerequisites → check capacity → confirm enrolment → update schedule and academic record." Patterns: Facade ⑥ · Strategy ① · Command ② · Observer ④



Facade ⑥ in one line
The Student sends exactly **one** message and receives one reply. Everything between messages 2 and 6 — validation strategy selection, command construction, transaction control, event publication — is hidden behind the facade. The Service API layer therefore contains no business rules.

Observer ④ — why message 5 is an open arrowhead
Message 5 is asynchronous and message 6 does not wait for it. The Facade holds no reference to any observer and never learns whether delivery succeeded. Observer fan-out is shown on page 2 (*Drop → Waitlist Notification*).

Command ② — the failure path
If step 2 or 3 inside execute() raises, TransactionManager pops its stack and calls undo() on every already-executed command in reverse order, then returns TxResult(failure). No partial state change survives. See diagram 04 (Activity) for the full rollback branch.

Sequence Diagram — Student Drops a Course → Automated Waitlist & Advisor Notification

Use case from §4: "A student drops a course. The notification system must automatically alert any waitlisted students that a spot has opened up. This process should be automated and **decoupled** from the core enrolment logic."

